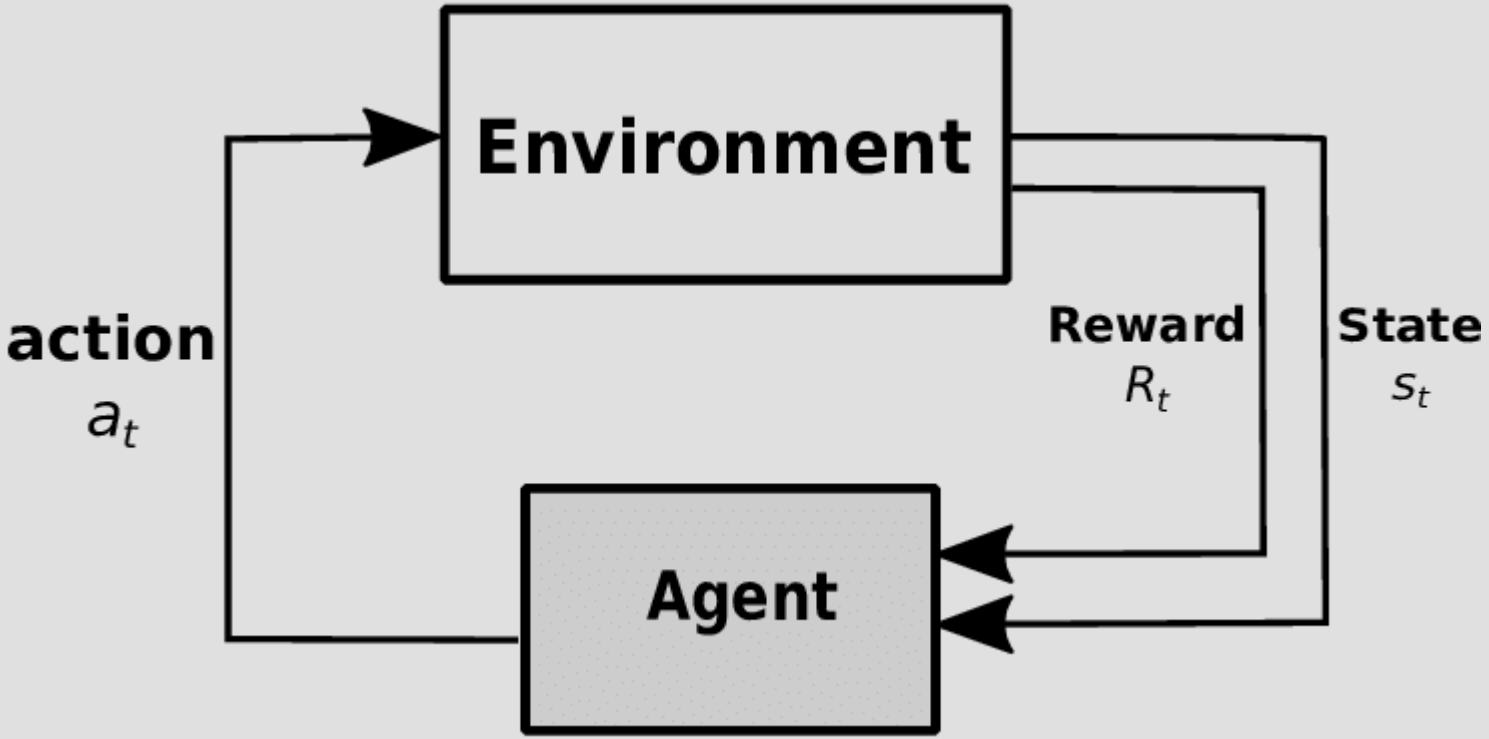
Reinforcement Learning II
Markov Decision Processes

Asad Jeewa
University of Cape Town



Recall: Components of an RL Framework



What will be covered

- What are Markov Decision Processes?
- Exploration vs Exploitation
- Models in RL

Formalising the RL problem

- Mathematical representation of the RL problem
  - State set
  - Transition function
  - Action set
  - Reward function
  - Discount factor

MDP Components

$$< S, A, T, R, \boldsymbol{\gamma} >$$

- S: Set of states i.e. encode environment configurations
- A: Set of actions i.e. the decisions made by the agent
- T: Transition function i.e. how the environment evolves under actions

$$T(s, a, s') = P(s_{t+1} = s' | s_t = s, a_t = a)$$

- R: Reward function i.e. immediate feedback signal $R(s, a) = c$
- $\gamma \in (0, 1]$: discounting for future rewards

What is Markov?

- The Markov property means that the future depends only on the current state and action — not on the full history

Future is independent of the past, given the present.

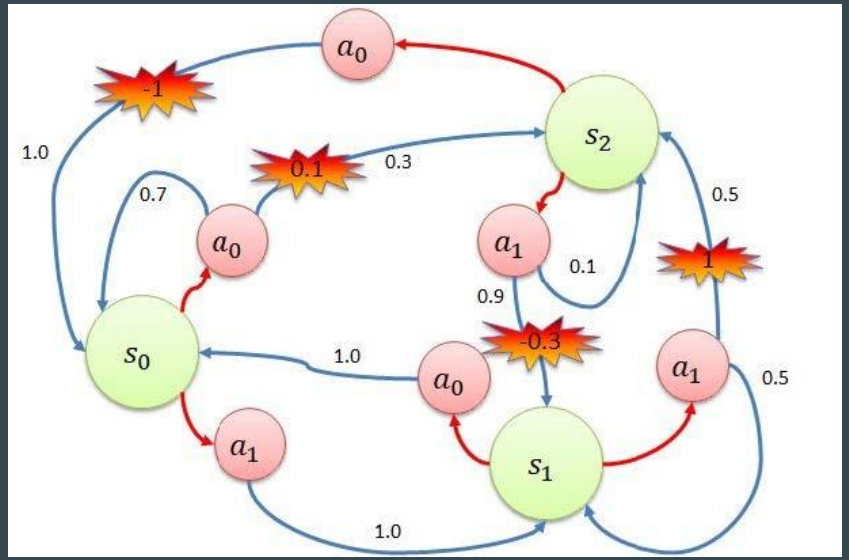
$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0, a_0) = P(s_{t+1} \mid s_t, a_t)$$

- Isn't this a major assumption?
- It is not something we "prove" about the real world; it is something we assume in order to make the problem tractable.
- S_t contains all relevant information from the past (not always the case)

Markov Models

- Markov Chain
  - A sequence of states that satisfies the Markov property
  - No actions or control
- Markov Decision Process
  - Extends a Markov chain by adding actions and rewards

MDP Components



A general example of an MDP showing S, A, R, T

Action spaces

$$< S, A, T, R, \boldsymbol{\gamma} >$$

- **Discrete**
  - Finite set of actions
  - E.g. Move Left / Move Right / Jump
- **Continuous**
  - Real-valued actions
  - E.g. Steering angle [-1,1], Throttle [0,1]

Observation/ State Spaces

$$< S, A, T, R, \gamma >$$

- **Discrete**
  - Finite number of states
  - E.g. Gridworld positions

- **Continuous**
  - Real-valued vectors
  - E.g. [x_position, y_position, velocity]

- **High-dimensional**
  - Images or sensor arrays
  - E.g. Camera frames in Atari / Robot sensors

Partial Observability

- **Fully Observable (MDP)**
  - Agent sees complete state
  - s_t = full environment snapshot

- **Partially Observable (POMDP)**
  - Agent sees only observation o_t
  - Must infer hidden state from history

- s_t represents the full environment information at time t.
- o_t is what the agent actually sees, which may be a partial view of s_t.
- E.g. a self-driving car sees only nearby objects (o_t) and not the full world around it

What is gamma?

$$G_0[\sum_{t=1}^{\infty}(\gamma^t r\,(s_t\,, a_t\,)]$$

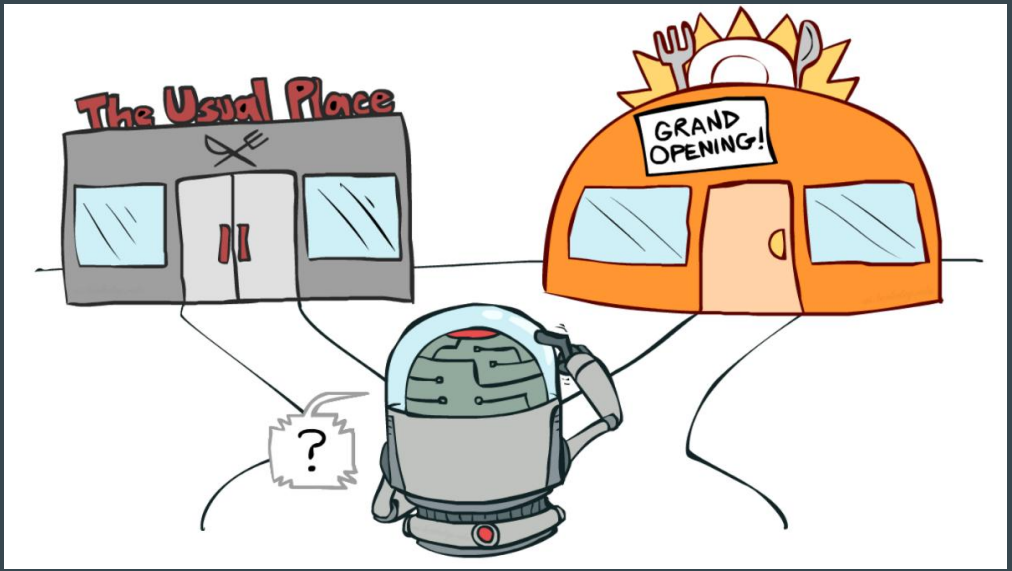
- Models preference for immediate vs delayed rewards
- Each reward is discounted once per time step it lies in the future,

- $\boldsymbol{\gamma} \rightarrow 0$: Agent is ‘myopic’ - focusing on immediate reward.
- $\boldsymbol{\gamma} \rightarrow 1$: Agent is ‘far-sighted’ - valuing future rewards almost as much as immediate rewards.
- Typically, $\gamma > 0.9$ to ensure future rewards are valued almost as highly as immediate rewards, encouraging long-term planning.

Benefits of Gamma

- Mathematical benefits
  - Ensures the cumulative return is finite
  - Guarantees convergence of value functions
  - Supports stability of dynamic programming methods
- Reduces the impact of long-term uncertainty
- Makes learning more stable

Exploration vs Exploitation



Exploration vs Exploitation

Exploration

- Goal: Discover new information about the environment.
- Agent tries out new and potentially unknown action or strategies.

Exploitation

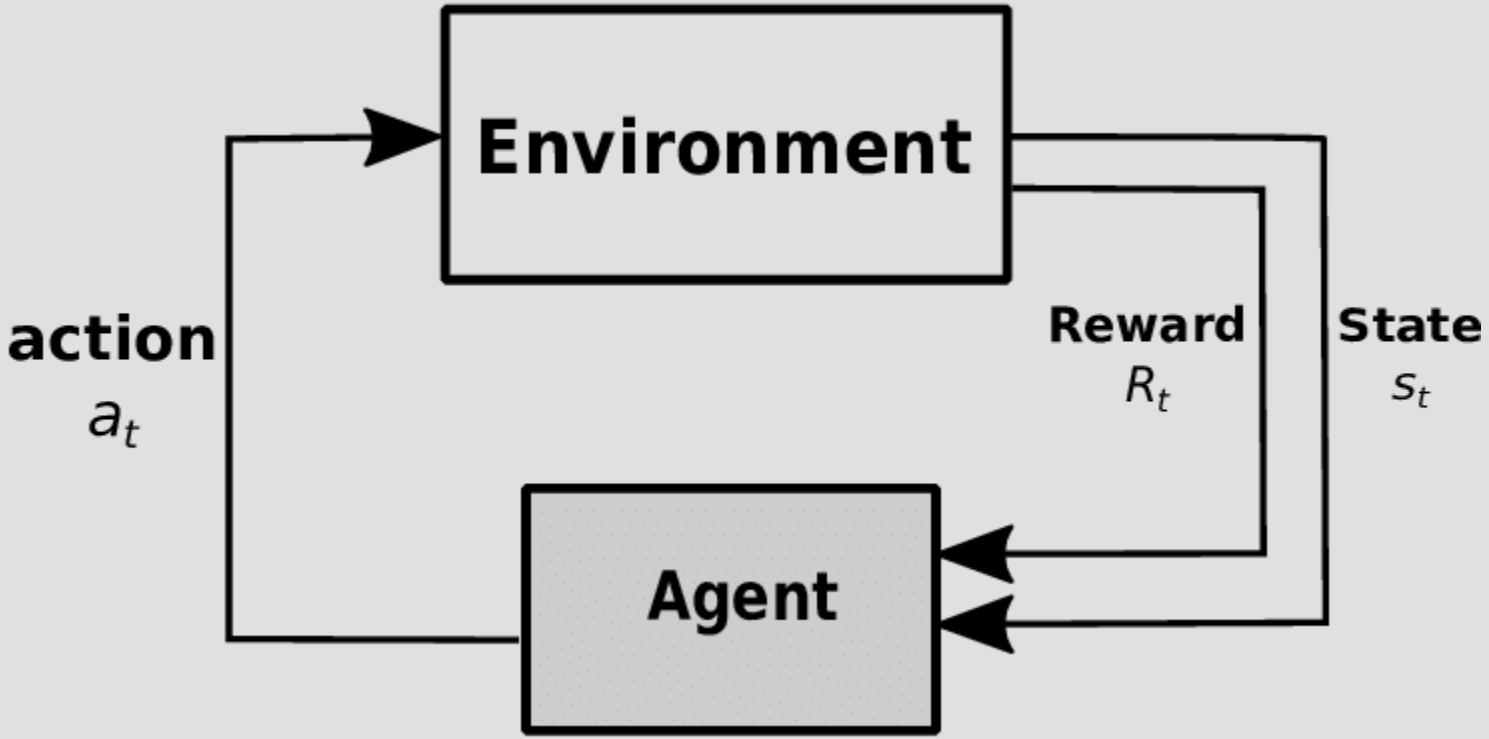
- Goal: To maximize the cumulative reward.
- Agent uses the knowledge it has already gained to take action that are know to yield high rewards, based on past experience.

- In RL we need to balance exploration with exploitation
  - Too much exploitation → local optima
  - Too much exploration → slow learning.

Common Approaches

- ε-greedy / Softmax: Randomly select actions to avoid getting stuck in known policies.
- Random Perturbations: Add noise to actions to encourage the agent to try new options.
- Uncertainty Estimation: Use model confidence to guide exploration where the agent is unsure.
- Intrinsic Motivation / Curiosity: Reward the agent for discovering novel or surprising states.

Components of an RL Framework



Goal

- Determine a policy
- Such that the cumulative (discounted) reward is maximised.

How?

- Predictions?
- Time-horizons?

Cumulative Discounted Return

- Goal: Maximise the expected return from a current state.
- Definition: Discounted sum of rewards from time step t to horizon H.

$$G_t = \sum_{k=0}^{H} \gamma^k R_{t+k+1}$$

(rewards are observed after the action, not at the same time step)

- Horizon
  - Number of steps over which the agent operates.
  - Finite vs infinite.

Model

- Represents the agent's internal understanding of the environment.
- Predicts:
  - Next state: $\hat{s}_{+1t} \approx s_{t+1}$ (approximate T in MDP)
  - Reward: $\hat{r}_t \approx r_t$ (approximate R in MDP)

Model-based vs Model-free RL

Model-free RL

- Given: S, A, R (States, Actions, Rewards only)
- Goal: Learn policy directly from experience
- Challenges:
  - Requires many interactions with the environment
  - Can be sample inefficient

Model-based RL

- Given: S, A, T, R (States, Actions, Transitions, Rewards)
- Goal: Learn or use a model of the environment to derive a policy.
- Simulate trajectories before acting → plan better actions
- Can reduce real environment interactions
- Challenges:
  - T and R may not be known and hence need to be learned.
  - Model learning errors can affect planning

Use Cases

Model-Based RL

- Use when: You can learn or know the environment model and planning is possible.
- Example: Chess: The rules are known, so the agent can plan ahead using the transition and reward model.

Model-Free RL

- Use when: The environment dynamics are unknown or too complex to model.
- Example: Recommender Systems (e.g. Netflix): Agent learns which recommendations lead to clicks/views and doesn't know the "model" of user behavior beforehand.

Resources

- Sutton & Barto (1998), Reinforcement Learning: An Introduction, Chapter 3.